

Upwork Data Analysis & Budget/Skills Forecasting

Data Scrapping From Upwork Using Selenium

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.chrome.options import Options
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
import time
import pandas as pd

base_url = "https://www.upwork.com/nx/search/jobs/?
from_recent_search=true&page={}&per_page=50"

job_data = []

for page_number in range(1, 26):
    driver = webdriver.Chrome()

    driver.get(base_url.format(page_number))
    time.sleep(5) # Allowing time for the page to load

    try:
        WebDriverWait(driver, 20).until(
            EC.presence_of_element_located((By.XPATH,
            '//*[@id="main"]/div/div/div/div[2]/div[2]/section/article[1]'))
        )
    except Exception as e:
        print(f"Error loading job listings on page {page_number}:", e)
        driver.quit()
        continue

    jobs = driver.find_elements(By.XPATH,
    '//*[@id="main"]/div/div/div/div[2]/div[2]/section/article')

    if not jobs:
        print(f"No jobs found on page {page_number}.")
        driver.quit()
        continue

    print(f"Scraping Page {page_number} with {len(jobs)} jobs found.")
    for job in jobs:
        try:
```

```

        title = job.find_element(By.XPATH, '//*[@h2/a]').text

        try:
            category = job.find_element(By.XPATH, '//*[@span[@data-
test="job-category"]]').text
        except:
            category = "Not specified"

        description = job.find_element(By.XPATH,
'//*[@p[contains(@class, "text-body-sm")]]').text

        try:
            skills_elements = job.find_elements(By.XPATH,
'//*[@button[contains(@class, "air3-token")]]')
            skills = ', '.join([skill.text for skill in
skills_elements])
        except:
            skills = "Not specified"

        try:
            budget = job.find_element(By.XPATH,
'//*[@span[contains(@class, "budget")]]').text
        except:
            budget = "Not specified"

        try:
            estimated_budget = job.find_element(By.XPATH,
'//*[@strong[2]]').text
        except:
            estimated_budget = "Not specified"

        try:
            client_location = job.find_element(By.XPATH,
'//*[@span[@data-test="client-location"]]').text
        except:
            client_location = "Not specified"

        try:
            client_spent = job.find_element(By.XPATH,
'//*[@span[@data-test="client-spent"]]').text
        except:
            client_spent = "Not specified"

        try:
            proposals = job.find_element(By.XPATH, '//*[@span[@data-
test="proposals-sent"]]').text
        except:
            proposals = "Not specified"

        try:

```

```

        experience_level = job.find_element(By.XPATH,
'./li[@data-test="experience-level"]').text
    except:
        experience_level = "Not specified"

        posted_time = job.find_element(By.XPATH,
'./small/span[2]').text

        try:
            job_type = job.find_element(By.XPATH, './li[@data-
test="job-type-label"]').text
        except:
            job_type = "Not specified"

    job_data.append({
        "Job Title": title,
        "Category": category,
        "Description": description,
        "Skills": skills,
        "Budget": budget,
        "Estimated Budget": estimated_budget,
        "Client Location": client_location,
        "Client Spent": client_spent,
        "Proposals": proposals,
        "Experience Level": experience_level,
        "Posted": posted_time,
        "Job Type": job_type
    })

except Exception as e:
    print(f"Error scraping job data: {e}")

driver.quit()

df = pd.DataFrame(job_data)

print(df)

df.to_csv('ProjectData1.csv', index=False, encoding='utf-8')
print("Data saved to ProjectData1.csv")

Scraping Page 1 with 50 jobs found.
Scraping Page 2 with 50 jobs found.
Scraping Page 3 with 50 jobs found.
Scraping Page 4 with 50 jobs found.
Scraping Page 5 with 50 jobs found.
Scraping Page 6 with 50 jobs found.
Scraping Page 7 with 50 jobs found.
Scraping Page 8 with 50 jobs found.

```

Scraping Page 9 with 50 jobs found.
 Scraping Page 10 with 50 jobs found.
 Scraping Page 11 with 50 jobs found.
 Scraping Page 12 with 50 jobs found.
 Scraping Page 13 with 50 jobs found.
 Scraping Page 14 with 50 jobs found.
 Scraping Page 15 with 50 jobs found.
 Scraping Page 16 with 50 jobs found.
 Scraping Page 17 with 50 jobs found.
 Scraping Page 18 with 50 jobs found.
 Scraping Page 19 with 50 jobs found.
 Scraping Page 20 with 50 jobs found.
 Scraping Page 21 with 50 jobs found.
 Scraping Page 22 with 50 jobs found.
 Scraping Page 23 with 50 jobs found.
 Scraping Page 24 with 50 jobs found.
 Scraping Page 25 with 50 jobs found.

	Job Title	Category
\		
0	Facebook Ads Specialist to Drive Lead Generati...	Not specified
1	Trading Signals Needed (Stocks / Forex / Crypto)	Not specified
2	Instagram Expert	Not specified
3	Graphic Designer Needed for Sports Posters	Not specified
4	Image Editing Expert for Website Photos and Co...	Not specified
...	...	...
1245	Custom Kiosk Display Design for Smart Toilet S...	Not specified
1246	List Building For Leads	Not specified
1247	Speech Recording Project - French	Not specified
1248	MT5 Trade Copying API Development (Single Term...	Not specified
1249	Build an iOS Widget and Integrate with existin...	Not specified

	Description	\
0	We are looking for an experienced Facebook Ads...	
1	Hi. I am looking for an expert that can share ...	
2	I need someone to build up my social media fol...	
3	We are looking for a talented graphic designer...	
4	We are looking for an experienced image editin...	
...	...	...
1245	We are seeking a talented designer to create a...	

1246 I'm looking for a list of restoration companie...
 1247 We are looking for native speakers to particip...
 1248 Requirements Copy trades from Master MT5 accou...
 1249 We're building Shelf getashelf.com: a combinat...

	Skills	Budget
\		
0	Appointment Setting, Lead Generation, Facebook	Not specified
1	Financial Analysis, Forex Trading, Cryptocurre...	Not specified
2	Instagram, Social Media Marketing, Instagram S...	Not specified
3	Graphic Design, Poster, Adobe Photoshop, Adobe...	Not specified
4	Photo Editing, Photo Retouching, Image Editing	Not specified
...	...	...

1245 Graphic Design, 3D Modeling, 3D Design, Produc... Not specified
 1246 Lead Generation, Data Entry, List Building, Co... Not specified
 1247 English, French, Female, Writing, General Tran... Not specified
 1248 Forex Trading, Bot Development, Foreign Exchan... Not specified
 1249 API Integration, Mobile App Development, Nativ... Not specified

	Estimated Budget	Client Location	\
0	More than 6 months, 30+ hrs/week	Not specified	
1	3 to 6 months, Less than 30 hrs/week	Not specified	
2	\$200.00	Not specified	
3	More than 6 months, Less than 30 hrs/week	Not specified	
4	1 to 3 months, Not sure	Not specified	
...	...	...	
1245	1 to 3 months, Less than 30 hrs/week	Not specified	
1246	Less than 1 month, Less than 30 hrs/week	Not specified	
1247	Less than 1 month, Less than 30 hrs/week	Not specified	
1248	Less than 1 month, Less than 30 hrs/week	Not specified	
1249	1 to 3 months, Less than 30 hrs/week	Not specified	

	Client Spent	Proposals	Experience Level	Posted	\
0	Not specified	Not specified	Intermediate	tomorrow	
1	Not specified	Not specified	Expert	tomorrow	
2	Not specified	Not specified	Expert	tomorrow	
3	Not specified	Not specified	Intermediate	tomorrow	
4	Not specified	Not specified	Intermediate	tomorrow	
...	...	...	...	...	
1245	Not specified	Not specified	Intermediate	in 1 hour	

1246	Not specified	Not specified	Intermediate	in 1 hour
1247	Not specified	Not specified	Intermediate	in 1 hour
1248	Not specified	Not specified	Intermediate	in 1 hour
1249	Not specified	Not specified	Expert	in 1 hour

	Job Type
0	Hourly: \$10.00 - \$50.00
1	Hourly
2	Fixed price
3	Hourly: \$5.00 - \$30.00
4	Hourly
...	...
1245	Hourly: \$15.00 - \$35.00
1246	Hourly
1247	Hourly
1248	Hourly
1249	Hourly: \$15.00 - \$40.00

[1250 rows x 12 columns]
Data saved to ProjectData1.csv

Data Preprocessing

Data Cleaning

```
import pandas as pd
import re
import unicodedata

file_path = 'a.csv'
data = pd.read_csv(file_path)

def clean_text(text, lowercase=True, remove_punctuation=False):
    if isinstance(text, str):
        text = text.encode('ascii', 'ignore').decode('ascii')

        text = re.sub(r'http\S+|www\S+', '', text)

        text = re.sub(r'\.{2,}', '.', text)

        text = ''.join(c for c in unicodedata.normalize('NFD', text)
                        if unicodedata.category(c) != 'Mn')

        if remove_punctuation:
            text = re.sub(r'[^\w\s]', '', text)  # Removes all
punctuation
```

```

        text = text.strip()

        if lowercase:
            text = text.lower()

        return text
    return text

def assign_category(description):
    if pd.isna(description):
        return 'Unspecified'

    description = str(description).lower()

    if any(word in description for word in ['web', 'website',
'frontend', 'backend']):
        return 'Web Development'
    elif any(word in description for word in ['data', 'analytics',
'machine learning', 'ai', 'big data']):
        return 'Data Science'
    elif any(word in description for word in ['marketing', 'seo',
'social media', 'adwords', 'advertising']):
        return 'Digital Marketing'
    elif any(word in description for word in ['design', 'graphic',
'ui', 'ux', 'logo', 'illustration']):
        return 'Design'
    elif any(word in description for word in ['app', 'mobile', 'ios',
'android']):
        return 'Mobile App Development'
    elif any(word in description for word in ['writing', 'content',
'blog', 'copywriting']):
        return 'Content Writing'
    elif any(word in description for word in ['video', 'editing',
'animation']):
        return 'Video Production'
    elif any(word in description for word in ['consulting',
'business', 'management']):
        return 'Business Consulting'
    elif any(word in description for word in ['customer service',
'support']):
        return 'Customer Support'
    else:
        return 'Other'

data['Description'] = data['Description'].apply(lambda x:
clean_text(x, lowercase=True, remove_punctuation=True))
data['Job Title'] = data['Job Title'].apply(lambda x: clean_text(x,
lowercase=True, remove_punctuation=True))

data['Category'] = data['Description'].apply(assign_category)

```

```

data['Skills'] = data['Skills'].apply(lambda x: [skill.strip() for
skill in x.split(',')]) if isinstance(x, str) else [])

data['Experience Level'] = data['Experience Level'].apply(lambda x:
clean_text(x, lowercase=True))
data['Job Type'] = data['Job Type'].apply(lambda x: clean_text(x,
lowercase=True))
data['Estimated Budget'] = data['Estimated Budget'].apply(lambda x:
clean_text(x, lowercase=True))
data['Client Location'] = data['Client Location'].apply(lambda x:
clean_text(x, lowercase=True))

def simplify_posted_time(value):
    if isinstance(value, str):
        if "hour" in value or "minute" in value:
            return "recent"
        elif "day" in value:
            return "a few days ago"
        elif "month" in value:
            return "a few months ago"
    return value

data['Posted'] = data['Posted'].apply(simplify_posted_time)

data.head()

{"summary":{"\n  \"name\": \"data\", \n  \"rows\": 1250, \n  \"fields\":
[\n    {\n      \"column\": \"Job Title\", \n      \"properties\": {\n
\"dtype\": \"string\", \n      \"num_unique_values\": 1165, \n
\"samples\": [\n        \"wordpress website design\", \n
\"real estate acquisition specialist\", \n        \"instagram
branding specialist for model marketing agency\" \n      ], \n
\"semantic_type\": \"\", \n      \"description\": \"\" \n    } \n
  ], \n    {\n      \"column\": \"Category\", \n      \"properties\":
{\n        \"dtype\": \"category\", \n        \"num_unique_values\":
10, \n        \"samples\": [\n          \"Mobile App Development\", \n
\"Data Science\", \n          \"Business Consulting\" \n        ], \n
\"semantic_type\": \"\", \n        \"description\": \"\" \n      } \n
  ], \n    {\n      \"column\": \"Description\", \n      \"properties\":
{\n        \"dtype\": \"string\", \n        \"num_unique_values\": 1157, \n
\"samples\": [\n          \"i have a single member llc and i need to file my personal tax return i
have detailed quickbooks records and taxes forms and just need an
accountant to prepare and file the forms for me\", \n          \"the
goal of the project is to enhance the websites user functionality
security and performance while keeping the existing graphics and
layout the current site though functional needs a full rebuild to meet
modern standards for security seo accessibility and performance key

```


tasks include upgrade php to version 8.3 or higher replace bootstrap 4 with pure css and update javascript upgrade the authorize.net checkout api and switch from font awesome to svg graphics optimize the database by improving indexing separating large tables and archiving inactive data enhance the image upload process converting uploads to webp and adding support for heif photos

"are you a motivated results-driven sales professional with a passion for personal development we are seeking dynamic commission-based sales representatives to promote and sell our highly effective procrastination coaching programs as part of our team you'll be responsible for introducing individuals and organizations to our transformative programs helping clients overcome procrastination and unlock their full potential this role offers a flexible schedule remote work and uncapped earning potential you'll earn competitive commissions for each successful enrollment in one of our coaching packages including our flagship programs procrastination to productivity and balance blueprint 8 weeks to life harmony key responsibilities identify and reach out to potential clients who would benefit from procrastination coaching effectively communicate the value of our programs and customize your approach to meet client needs close sales and enroll clients in our coaching packages build and maintain relationships with clients to ensure long-term satisfaction and potential referrals what we offer uncapped commission structure comprehensive training on our programs and the benefits of procrastination coaching flexible remote working opportunities a chance to be part of a growing impactful coaching business that helps people transform their lives ideal candidate proven sales experience preferably in personal development coaching or related fields strong communication and interpersonal skills self-motivated with a passion for helping others improve their lives ability to work independently and hit sales targets join me in helping individuals break free from procrastination and live more productive fulfilling lives"

```

n      ],\n      \"semantic_type\": \"\",\n      \"description\": \"\"\n    },\n    {\n      \"column\": \"Skills\", \n      \"properties\": {\n        \"dtype\": \"object\", \n        \"semantic_type\": \"\", \n        \"description\": \"\"\n      },\n      {\n        \"column\": \"Budget\", \n        \"properties\": {\n          \"dtype\": \"category\", \n          \"num_unique_values\": 6, \n          \"samples\": [\n            \"$300 - $500\", \n            \"$500 - $1000\", \n            \"$5000 - $10000\" \n          ], \n          \"semantic_type\": \"\", \n          \"description\": \"\"\n        },\n        {\n          \"column\": \"Estimated Budget\", \n          \"properties\": {\n            \"dtype\": \"category\", \n            \"num_unique_values\": 83, \n            \"samples\": [\n              \"$50.00\", \n              \"1 to 3 months, less than 30 hrs/week\", \n              \"1 to 3 months, 30+ hrs/week\" \n            ], \n            \"semantic_type\": \"\", \n            \"description\": \"\"\n          },\n          {\n            \"column\": \"Client Location\", \n            \"properties\": {\n              \"dtype\": \"category\", \n    
```

```

{"num_unique_values": 10,\n      "samples": [\n        "united states",\n        "germany",\n        "australia",\n        "netherlands",\n        "semantic_type": "\n",\n        "description": "\n",\n        "column": "Client Spent",\n        "properties": {\n          "dtype": "category",\n          "num_unique_values": 5,\n          "samples": [\n            "$1000 - $5000",\n            "$5000 - $10000",\n            "$500 - $1000",\n            "semantic_type": "\n",\n            "description": "\n",\n            "column": "Proposals",\n            "properties": {\n              "dtype": "category",\n              "num_unique_values": 5,\n              "samples": [\n                "20 to 50",\n                "10 to 20",\n                "50+",\n                "semantic_type": "\n",\n                "description": "\n",\n                "column": "Experience Level",\n                "properties": {\n                  "dtype": "category",\n                  "num_unique_values": 3,\n                  "samples": [\n                    "expert",\n                    "intermediate",\n                    "entry level",\n                    "semantic_type": "\n",\n                    "description": "\n",\n                    "column": "Posted",\n                    "properties": {\n                      "dtype": "category",\n                      "num_unique_values": 6,\n                      "samples": [\n                        "recent",\n                        "a few months ago",\n                        "last week",\n                        "semantic_type": "\n",\n                        "description": "\n",\n                        "column": "Job Type",\n                        "properties": {\n                          "dtype": "category",\n                          "num_unique_values": 180,\n                          "samples": [\n                            "hourly: $4.00 - $7.00",\n                            "hourly: $30.00 - $50.00",\n                            "hourly: $4.00 - $8.00",\n                            "semantic_type": "\n",\n                            "description": "\n",\n                            "column": " ]\n      ]\n    },\n    "type": "dataframe",\n    "variable name": "data"}

```

Extracting numerical values fro each column

```
import pandas as pd
import numpy as np

def client_spent_to_median(spent_range):
    if '+' in spent_range:
        return 10000
    elif 'Less than' in spent_range:
        return 250
    elif '-' in spent_range:
        lower, upper = spent_range.replace('$', '').split(' - ')
        return np.median([int(lower), int(upper)])
    else:
        return int(spent_range.replace('$', '').replace('Less than ', '').strip())
```

```

def proposals_to_max(proposal_range):
    if 'Less than' in proposal_range:
        return 4
    elif '+' in proposal_range:
        return
    elif 'to' in proposal_range:
        lower, upper = proposal_range.replace(' to ', ' ').split(' ')
        return int(upper)
    else:
        return int(proposal_range)

data['Proposals'] = data['Proposals'].apply(proposals_to_max)

data['Client Spent'] = data['Client
Spent'].apply(client_spent_to_median)

def clean_job_type(job_type):
    if 'hourly' in job_type:
        return 'hourly'
    elif 'fixed price' in job_type:
        return 'fixed price'
    else:
        return job_type

data['Job Type'] = data['Job Type'].apply(clean_job_type)

```

Dropping Unnecessary columns which are not required

```

data=data.drop(columns='Estimated Budget')
data=data.drop(columns='Posted')
data.head()

{"summary":{"\n  \"name\": \"data\",\n  \"rows\": 1250,\n  \"fields\": [\n    {\n      \"column\": \"Job Title\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 1165,\n        \"samples\": [\n          \"wordpress website design\",\n          \"real estate acquisition specialist\",\n          \"instagram branding specialist for model marketing agency\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Category\",\n      \"properties\": {\n        \"dtype\": \"category\",\n        \"num_unique_values\": 10,\n        \"samples\": [\n          \"Mobile App Development\",\n          \"Data Science\",\n          \"Business Consulting\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Description\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 1157,\n        \"samples\": [\n          \"i

```

I have a single member llc and i need to file my personal tax return I have detailed quickbooks records and taxes forms and just need an accountant to prepare and file the forms for me

"the goal of the project is to enhance the websites user functionality security and performance while keeping the existing graphics and layout the current site though functional needs a full rebuild to meet modern standards for security seo accessibility and performance key tasks include upgrade php to version 8.3 or higher replace bootstrap 4 with pure css and update javascript upgrade the authorize.net checkout api and switch from font awesome to svg graphics optimize the database by improving indexing separating large tables and archiving inactive data enhance the image upload process converting uploads to webp and adding support for heif photos"

"are you a motivated results-driven sales professional with a passion for personal development we are seeking dynamic commission-based sales representatives to promote and sell our highly effective procrastination coaching programs as part of our team you'll be responsible for introducing individuals and organizations to our transformative programs helping clients overcome procrastination and unlock their full potential this role offers a flexible schedule remote work and uncapped earning potential you'll earn competitive commissions for each successful enrollment in one of our coaching packages including our flagship programs procrastination to productivity and balance blueprint 8 weeks to life harmony key responsibilities identify and reach out to potential clients who would benefit from procrastination coaching effectively communicate the value of our programs and customize your approach to meet client needs close sales and enroll clients in our coaching packages build and maintain relationships with clients to ensure long-term satisfaction and potential referrals what we offer uncapped commission structure comprehensive training on our programs and the benefits of procrastination coaching flexible remote working opportunities a chance to be part of a growing impactful coaching business that helps people transform their lives ideal candidate proven sales experience preferably in personal development coaching or related fields strong communication and interpersonal skills self-motivated with a passion for helping others improve their lives ability to work independently and hit sales targets join me in helping individuals break free from procrastination and live more productive fulfilling lives"

```
[{"description": "Procrastination Coaching Programs", "column": "Budget", "properties": {"dtype": "number", "std": 195560.47881810283, "min": 1.0, "max": 2000000.0, "num_unique_values": 572, "samples": [2794.9860000000003, 1521.3572, 2068.5010500000008]}}, {"semantic_type": ""}], [{"description": "", "column": "Budget", "properties": {"dtype": "object", "std": null, "min": null, "max": null, "num_unique_values": null, "samples": []}}], [{"description": "Skills", "column": "Properties", "properties": {"dtype": "object", "std": null, "min": null, "max": null, "num_unique_values": null, "samples": []}}, {"semantic_type": "", "description": "", "column": "Description", "properties": {}}, {"column": "Column", "properties": {"dtype": "number", "std": null, "min": null, "max": null, "num_unique_values": null, "samples": []}}, {"semantic_type": ""}]]]
```

```

{"description": "", "column": "Client Location", "properties": {"num_unique_values": 10, "samples": [{"united states", "germany", "australia"}], "semantic_type": "category", "dtype": "category"}, {"description": "Client Spent", "column": "Client Spent", "properties": {"std": 3754.7178169215217, "min": 250.0, "max": 10000.0, "num_unique_values": 5, "samples": [3000.0, 7500.0, 750.0]}, "semantic_type": "number", "dtype": "float64"}, {"description": "Proposals", "column": "Proposals", "properties": {"std": 19, "min": 4, "max": 50, "num_unique_values": 3, "samples": [50, 10, 4]}, "semantic_type": "number", "dtype": "float64"}, {"description": "Experience Level", "column": "Experience Level", "properties": {"dtype": "category", "num_unique_values": 3, "samples": ["intermediate", "expert", "entry"]}, "semantic_type": "category", "dtype": "category"}, {"description": "Job Type", "column": "Job Type", "properties": {"dtype": "category", "num_unique_values": 2, "samples": ["fixed price", "hourly"]}, "semantic_type": "category", "dtype": "category"}], "type": "dataframe", "variable_name": "data"}

```

Dropping empty skills rows

```
data = data[data['Skills'].apply(lambda x: len(x) > 0)]
```

Checking Nulls. No nulls found

```
data.isnull().sum()
```

```

Job Title      0
Category       0
Description     0
Skills         0
Budget         0
Client Location 0
Client Spent   0
Proposals      0
Experience Level 0
Job Type       0
dtype: int64

```

Feature Engineering

Making 4 new columns from the data to increase Data accuracy.

Cost per Skill: This feature provides insight into how much budget is allocated for each required skill. A lower cost per skill could indicate that the client seeks more general skills or has budget constraints, while a higher cost per skill might imply a preference for specialized skills.

Proposal-to-Budget Ratio: This ratio gauges the job's competitiveness by comparing the budget to the number of proposals received. A low ratio suggests a high number of proposals for a modest budget, potentially indicating job popularity, while a high ratio might suggest exclusivity or complexity due to fewer proposals or a high budget.

Client Spending Frequency: By dividing the total amount spent by the client by the number of proposals, this feature offers insight into how frequently or consistently the client hires. It can reveal client reliability—clients with a high spending frequency might be more active or higher-budget spenders, useful for prioritizing projects.

Budget Grouping: Bin the Budget column into ranges (e.g., Low, Medium, High) to simplify budget categories, which can help analyze job trends based on budget groups.

```
import pandas as pd

data['Skills'] = data['Skills'].apply(
    lambda x: ','.join(x) if isinstance(x, list) else x
)

data['cost_per_skill'] = data.apply(
    lambda row: row['Budget'] / len(row['Skills'].split(', '))
    if isinstance(row['Skills'], str) and len(row['Skills'].strip()) >
0 else 0,
    axis=1
)

data['proposal_to_budget_ratio'] = data.apply(lambda row:
row['Budget'] / row['Proposals'] if row['Proposals'] > 0 else 0,
axis=1)

data['client_spending_frequency'] = data.apply(lambda row: row['Client
Spent'] / row['Proposals'] if row['Proposals'] > 0 else 0, axis=1)

data['budget_group'] = pd.cut(data['Budget'], bins=[0, 500, 5000,
10000], labels=['Low', 'Medium', 'High'])

data.head()

{"summary": "{\n  \"name\": \"data\", \n  \"rows\": 1237, \n  \"fields\": [\n    {\n      \"column\": \"Job Title\", \n      \"properties\": {\n        \"dtype\": \"string\", \n        \"num_unique_values\": 1153, \n
```

```

\"samples\": [\n          \"high quality product photography vitamin  
bottle spoon that can be used in digital advertising\", \n  

\"google analytics reporting specialist needed\", \n  

\"prospect generation for niche software\", \n          ], \n  

\"semantic_type\": \"\", \n          \"description\": \"\" \n          } \n  

n      }, \n      { \n          \"column\": \"Category\", \n          \"properties\":  
{ \n          \"dtype\": \"category\", \n          \"num_unique_values\":  
10, \n          \"samples\": [\n          \"Mobile App Development\", \n  

\"Data Science\", \n          \"Business Consulting\", \n          ], \n  

\"semantic_type\": \"\", \n          \"description\": \"\" \n          } \n  

n      }, \n      { \n          \"column\": \"Description\", \n  

\"properties\": { \n          \"dtype\": \"string\", \n  

\"num_unique_values\": 1145, \n          \"samples\": [\n          \"we  
are seeking a skilled cold calling specialist to help us introduce our  
services and set up meetings with potential clients the ideal  
candidate will have experience in making outbound calls engaging  
prospects and securing appointments you will be responsible for  
creating a positive first impression and ensuring our message  
resonates with the target audience if you have excellent communication  
skills and a track record in successful cold calling we want to hear  
from you relevant skills cold calling appointment setting lead  
generation excellent communication skills persuasion and negotiation  
skills will pay per meeting set up 10 usd only apply if ok\", \n  

\"we are seeking an experienced react developer to build a dynamic  
single page application spa that provides a seamless user experience  
the ideal candidate should be comfortable with componentbased  
architecture and have a strong understanding of state management and  
api integration your role will involve collaborating with our design  
and backend teams to transform wireframes into highquality code if you  
have a passion for developing engaging web applications and a keen eye  
for detail we want to hear from you\", \n          \"we are seeking a  
talented graphic designer and writer to create a visually appealing  
tweet graphic for our recently accepted academic paper at a  
prestigious conference the ideal candidate will transform complex  
research findings into an engaging and informative design that  
captures attention on social media a quick turnaround and creativity  
are essential as well as understanding academic themes if you have a  
passion for science communication and experience in creating impactful  
visuals we would love to hear from you\", \n          ], \n  

\"semantic_type\": \"\", \n          \"description\": \"\" \n          } \n  

n      }, \n      { \n          \"column\": \"Skills\", \n          \"properties\":  
{ \n          \"dtype\": \"string\", \n          \"num_unique_values\":  
1137, \n          \"samples\": [\n          \"PyTorch,Cloud Computing\", \n  

n          \"Social Media Marketing,Communications,Email  
Communication,Instagram,Facebook,Administrative Support\", \n  

\"Data Entry,Amazon PPC,SEO Backlinking,Amazon FBA,PPC Campaign Setup  
& Management,Dropshipping\", \n          ], \n          \"semantic_type\":  
\"\", \n          \"description\": \"\" \n          } \n      }, \n      { \n  

\"column\": \"Budget\", \n          \"properties\": { \n          \"dtype\":

```



```
\ "number\\",\n      \ "std\\": 196565.2137141231,\n      \ "min\\": 1.0,\n      \n      \ "max\\": 2000000.0,\n      \n      \ "num_unique_values\\": 571,\n      \n      \ "samples\\": [\n          2716.254,\n          1521.3572,\n          2068.50105000000008\n      ],\n      \n      \ "semantic_type\\": \ "","description\\": \ "","column\\": \ "Client Location\\",\n      \n      \n      \n      \n      \ "properties\\": {\n          \ "dtype\\": \ "category\\",\n          \ "num_unique_values\\": 10,\n          \ "samples\\": [\n              \ "united states\\",\n              \ "germany\\",\n              \ "australia\\",\n              \n          ],\n          \ "semantic_type\\": \ "","description\\": \ "","column\\": \ "Client Spent\\",\n          \n      },\n      \n      \n      \n      \n      \ "properties\\": {\n          \ "dtype\\": \ "number\\",\n          \ "std\\": 3757.5548256872944,\n          \ "min\\": 250.0,\n          \ "max\\": 10000.0,\n          \ "num_unique_values\\": 5,\n          \ "samples\\": [\n              3000.0,\n              7500.0,\n              750.0\n          ],\n          \ "semantic_type\\": \ "","description\\": \ "","column\\": \ "Proposals\\",\n          \n      },\n      \n      \n      \n      \n      \ "properties\\": {\n          \ "dtype\\": \ "number\\",\n          \ "std\\": 19,\n          \ "min\\": 4,\n          \ "max\\": 50,\n          \ "num_unique_values\\": 3,\n          \ "samples\\": [\n              50,\n              10,\n              4\n          ],\n          \ "semantic_type\\": \ "","description\\": \ "","column\\": \ "Experience Level\\",\n          \n      },\n      \n      \n      \n      \n      \ "properties\\": {\n          \ "dtype\\": \ "category\\",\n          \ "num_unique_values\\": 3,\n          \ "samples\\": [\n              \ "intermediate\\",\n              \ "expert\\",\n              \ "entry\\",\n              \n          ],\n          \ "semantic_type\\": \ "","description\\": \ "","column\\": \ "Job Type\\",\n          \n      },\n      \n      \n      \n      \n      \ "properties\\": {\n          \ "dtype\\": \ "category\\",\n          \ "num_unique_values\\": 2,\n          \ "samples\\": [\n              \ "fixed price\\",\n              \ "hourly\\",\n              \n          ],\n          \ "semantic_type\\": \ "","description\\": \ "","column\\": \ "cost_per_skill\\",\n          \n      },\n      \n      \n      \n      \n      \ "properties\\": {\n          \ "dtype\\": \ "number\\",\n          \ "std\\": 196565.2137141231,\n          \ "min\\": 1.0,\n          \ "max\\": 2000000.0,\n          \ "num_unique_values\\": 571,\n          \ "samples\\": [\n              2716.254,\n              1521.3572\n          ],\n          \ "semantic_type\\": \ "","description\\": \ "","column\\": \ "proposal_to_budget_ratio\\",\n          \n      },\n      \n      \n      \n      \n      \ "properties\\": {\n          \ "dtype\\": \ "number\\",\n          \ "std\\": 17602.919845458433,\n          \ "min\\": 0.02,\n          \ "max\\": 250000.0,\n          \ "num_unique_values\\": 580,\n          \ "samples\\": [\n              650.00847600000002,\n              587.9767725\n          ],\n          \ "semantic_type\\": \ "","description\\": \ "","column\\": \ "client_spending_frequency\\",\n          \n      },\n      \n      \n      \n      \n      \ "properties\\": {\n          \ "dtype\\": \ "number\\",\n          \ "std\\": 549.6376562346827,\n          \ "min\\": 5.0,\n          \ "max\\": 2500.0,\n          \ "num_unique_values\\": 14,\n          \ "samples\\": [\n              2500.0,\n              \n          ],\n          \ "semantic type\\": \ "","
```



```

\ "description\": \ "\n      }\n    },\n    {\n      \ "column\":
\ "budget_group\","\n      \ "properties\": {\n      \ "dtype\":
\ "category\","\n      \ "num_unique_values\": 2,\n      \ "samples\":
[\n      \ "Low\","\n      \ "Medium\","\n      ],\n
\ "semantic_type\": \ "\",\n      \ "description\": \ "\n      }\n
n      }\n    ]\n}", "type": "dataframe", "variable_name": "data"}

```

Checking outliers in the data and replacing them with median since our data is not so large

```

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

def fill_outliers_with_median(df, column):
    Q1 = df[column].quantile(0.25)
    Q3 = df[column].quantile(0.75)
    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    print(f"Lower bound for '{column}': {lower_bound}")
    print(f"Upper bound for '{column}': {upper_bound}")

    outliers = df[(df[column] < lower_bound) | (df[column] >
upper_bound)]
    num_outliers = len(outliers)

    print(f"Number of outliers in '{column}': {num_outliers}")

    median_value = df[column].median()
    df.loc[(df[column] < lower_bound) | (df[column] > upper_bound),
column] = median_value

fill_outliers_with_median(data, 'Budget')
fill_outliers_with_median(data, 'Proposals')
fill_outliers_with_median(data, 'Client Spent')
fill_outliers_with_median(data, 'cost_per_skill')
fill_outliers_with_median(data, 'proposal_to_budget_ratio')
fill_outliers_with_median(data, 'client_spending_frequency')

plt.figure(figsize=(10, 6))
sns.boxplot(data['Budget'])
plt.title('Budget Distribution Before Removing Outliers')
plt.show()

Q1 = data['Budget'].quantile(0.25)
Q3 = data['Budget'].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR

```

```

upper_bound = Q3 + 1.5 * IQR
data_no_outliers = data[(data['Budget'] >= lower_bound) &
(data['Budget'] <= upper_bound)]

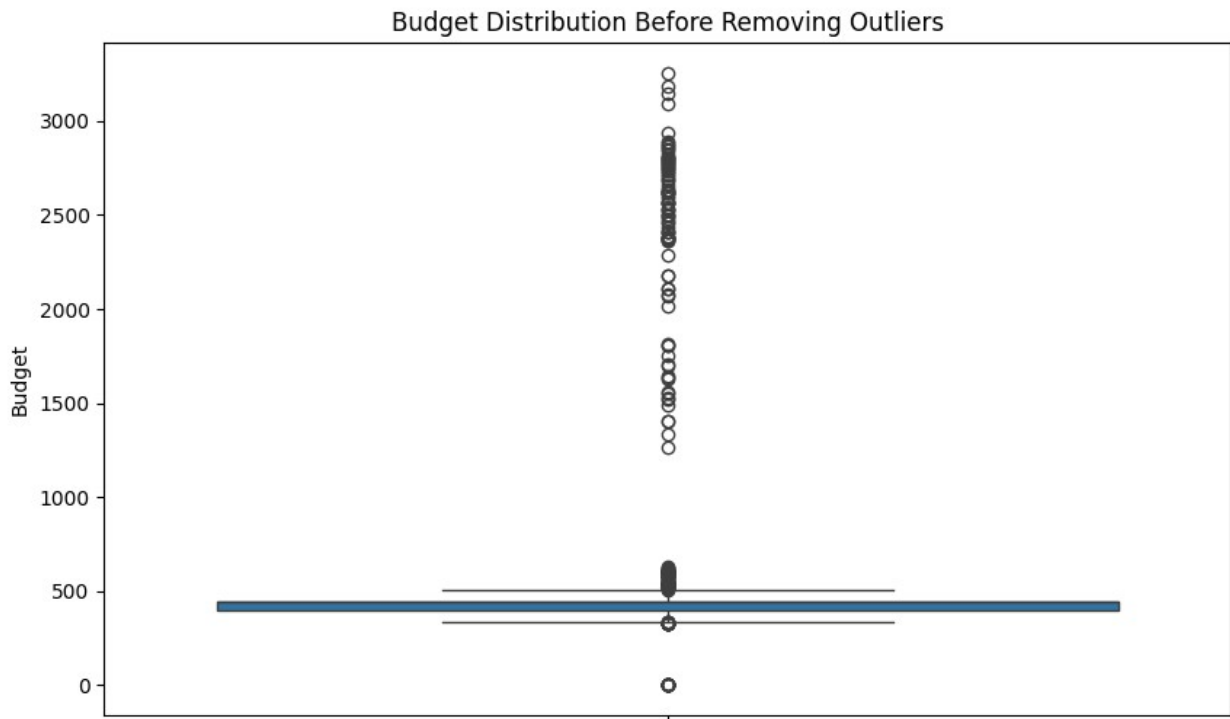
plt.figure(figsize=(10, 6))
sns.boxplot(data_no_outliers['Budget'])
plt.title('Budget Distribution After Removing Outliers (IQR Method)')
plt.show()

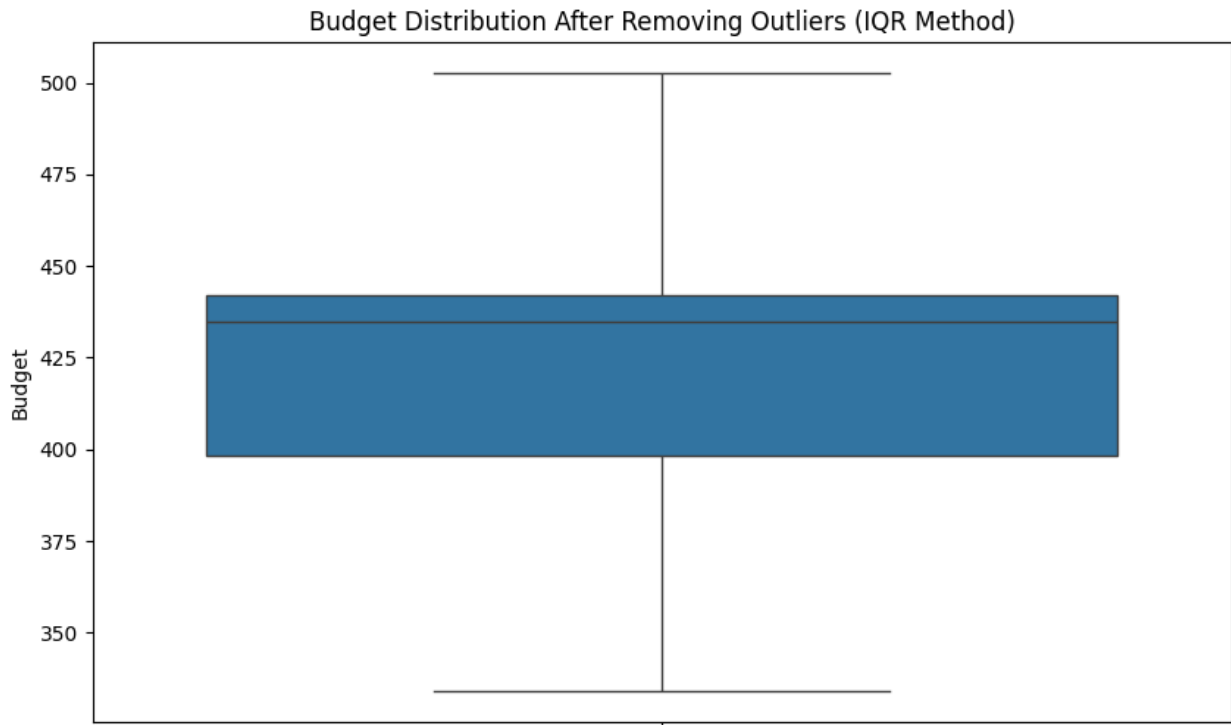
```

```

Lower bound for 'Budget': -3337.3019718750015
Upper bound for 'Budget': 6624.882703125002
Number of outliers in 'Budget': 304
Lower bound for 'Proposals': -50.0
Upper bound for 'Proposals': 110.0
Number of outliers in 'Proposals': 0
Lower bound for 'Client Spent': -9375.0
Upper bound for 'Client Spent': 17625.0
Number of outliers in 'Client Spent': 0
Lower bound for 'cost_per_skill': -3337.3019718750015
Upper bound for 'cost_per_skill': 6624.882703125002
Number of outliers in 'cost_per_skill': 304
Lower bound for 'proposal_to_budget_ratio': -40.780781250000004
Upper bound for 'proposal_to_budget_ratio': 179.83246875000003
Number of outliers in 'proposal_to_budget_ratio': 251
Lower bound for 'client_spending_frequency': -300.0
Upper bound for 'client_spending_frequency': 660.0
Number of outliers in 'client_spending_frequency': 304

```





```
data.head()
```

```
{
  "summary": {
    "name": "data",
    "rows": 1237,
    "fields": [
      {
        "column": "Job Title",
        "properties": {
          "dtype": "string",
          "num_unique_values": 1153,
          "samples": [
            "high quality product photography vitamin bottle spoon that can be used in digital advertising",
            "google analytics reporting specialist needed",
            "prospect generation for niche software"
          ],
          "semantic_type": "",
          "description": ""
        }
      },
      {
        "column": "Category",
        "properties": {
          "dtype": "category",
          "num_unique_values": 10,
          "samples": [
            "Mobile App Development",
            "Data Science",
            "Business Consulting"
          ],
          "semantic_type": "",
          "description": ""
        }
      },
      {
        "column": "Description",
        "properties": {
          "dtype": "string",
          "num_unique_values": 1145,
          "samples": [
            "we are seeking a skilled cold calling specialist to help us introduce our services and set up meetings with potential clients the ideal candidate will have experience in making outbound calls engaging prospects and securing appointments you will be responsible for creating a positive first impression and ensuring our message resonates with the target audience if you have excellent communication skills and a track record in successful cold calling we want to hear from you relevant skills cold calling appointment setting lead generation excellent communication skills persuasion and negotiation"
          ]
        }
      }
    ]
  }
}
```

skills will pay per meeting set up 10 usd only apply if ok",\n

"we are seeking an experienced react developer to build a dynamic single page application spa that provides a seamless user experience the ideal candidate should be comfortable with componentbased architecture and have a strong understanding of state management and api integration your role will involve collaborating with our design and backend teams to transform wireframes into highquality code if you have a passion for developing engaging web applications and a keen eye for detail we want to hear from you",\n

"we are seeking a talented graphic designer and writer to create a visually appealing tweet graphic for our recently accepted academic paper at a prestigious conference the ideal candidate will transform complex research findings into an engaging and informative design that captures attention on social media a quick turnaround and creativity are essential as well as understanding academic themes if you have a passion for science communication and experience in creating impactful visuals we would love to hear from you"\n

],\n

"semantic_type": "",\n

"description": ""\n

},\n

{\n

"column": "Skills",\n

"properties": {\n

"dtype": "string",\n

"num_unique_values": 1137,\n

"samples": [\n

"PyTorch,Cloud Computing",\n

"Social Media Marketing,Communications,Email Communication,Instagram,Facebook,Administrative Support",\n

"Data Entry,Amazon PPC,SEO Backlinking,Amazon FBA,PPC Campaign Setup & Management,Dropshipping"\n

],\n

"semantic_type": "",\n

"description": ""\n

},\n

{\n

"column": "Budget",\n

"properties": {\n

"dtype": "number",\n

"std": 538.4613409087323,\n

"min": 1.0,\n

"max": 3252.5064000000007,\n

"num_unique_values": 274,\n

"samples": [\n

419.04,\n

580.04285625,\n

1807.2126\n

],\n

"semantic_type": "",\n

"description": ""\n

},\n

{\n

"column": "Client Location",\n

"properties": {\n

"dtype": "category",\n

"num_unique_values": 10,\n

"samples": [\n

"united states",\n

"germany",\n

"australia"\n

],\n

"semantic_type": "",\n

"description": ""\n

},\n

{\n

"column": "Client Spent",\n

"properties": {\n

"dtype": "number",\n

"std": 3757.5548256873017,\n

"min": 250.0,\n

"max": 10000.0,\n

"num_unique_values": 5,\n

"samples": [\n

3000.0,\n

7500.0,\n

750.0\n

],\n

"semantic_type": "",\n

"description": ""\n

},\n

{\n

"column": "Proposals",\n

"properties": {\n

"dtype": "number",\n

"std": 19,\n

"min": 4,\n

"max": 50,\n

"num_unique_values": 3,\n

"samples": [\n

50,\n

10,\n

4\n

],\n

"semantic_type": "",\n

"description": ""\n

}\n

```

n    },\n    {\n        \"column\": \"Experience Level\",\n        \"properties\": {\n            \"dtype\": \"category\",\n            \"num_unique_values\": 3,\n            \"samples\": [\n                \"intermediate\",\n                \"expert\",\n                \"entry\"\n            ],\n            \"semantic_type\": \"\",\n            \"description\": \"\"\n        },\n        {\n            \"column\": \"Job Type\",\n            \"properties\": {\n                \"dtype\": \"category\",\n                \"num_unique_values\": 2,\n                \"samples\": [\n                    \"fixed price\",\n                    \"hourly\"\n                ],\n                \"semantic_type\": \"\",\n                \"description\": \"\"\n            },\n            {\n                \"column\": \"cost_per_skill\",\n                \"properties\": {\n                    \"dtype\": \"number\",\n                    \"std\": 538.4613409087323,\n                    \"min\": 1.0,\n                    \"max\": 3252.5064000000007,\n                    \"num_unique_values\": 274,\n                    \"samples\": [\n                        419.04,\n                        580.04285625\n                    ],\n                    \"semantic_type\": \"\",\n                    \"description\": \"\"\n                },\n                {\n                    \"column\": \"proposal_to_budget_ratio\",\n                    \"properties\": {\n                        \"dtype\": \"number\",\n                        \"std\": 22.313560182947423,\n                        \"min\": 0.02,\n                        \"max\": 175.39614,\n                        \"num_unique_values\": 274,\n                        \"samples\": [\n                            0.1,\n                            58.004285625\n                        ],\n                        \"semantic_type\": \"\",\n                        \"description\": \"\"\n                    },\n                    {\n                        \"column\": \"client_spending_frequency\",\n                        \"properties\": {\n                            \"dtype\": \"number\",\n                            \"std\": 87.18802472662703,\n                            \"min\": 5.0,\n                            \"max\": 300.0,\n                            \"num_unique_values\": 10,\n                            \"samples\": [\n                                5.0,\n                                60.0\n                            ],\n                            \"semantic_type\": \"\",\n                            \"description\": \"\"\n                        },\n                        {\n                            \"column\": \"budget_group\",\n                            \"properties\": {\n                                \"dtype\": \"category\",\n                                \"num_unique_values\": 2,\n                                \"samples\": [\n                                    \"Low\",\n                                    \"Medium\"\n                                ],\n                                \"semantic_type\": \"\",\n                                \"description\": \"\"\n                            }\n                        }\n                    ],\n                    \"type\": \"dataframe\", \"variable_name\": \"data\"}

```

Data Transformation

Applying Tfidf on columns [Job title, Description, Skills]

```

from sklearn.feature_extraction.text import TfidfVectorizer
import pandas as pd

title_vectorizer = TfidfVectorizer(max_features=500)
description_vectorizer = TfidfVectorizer(max_features=500)
skills_vectorizer = TfidfVectorizer(max_features=500)

title_tfidf = title_vectorizer.fit_transform(data['Job Title'])

```

```

description_tfidf =
description_vectorizer.fit_transform(data['Description'])
skills_tfidf = skills_vectorizer.fit_transform(data['Skills'])

title_df = pd.DataFrame(title_tfidf.toarray(),
columns=title_vectorizer.get_feature_names_out())
description_df = pd.DataFrame(description_tfidf.toarray(),
columns=description_vectorizer.get_feature_names_out())
skills_df = pd.DataFrame(skills_tfidf.toarray(),
columns=skills_vectorizer.get_feature_names_out())

```

Encoding Labels on columns [Client location, Category , Budget Type] and One Hot Encoding on [Experience Level & Job Type]

```

from sklearn.preprocessing import LabelEncoder

data['Client Location'] = LabelEncoder().fit_transform(data['Client
Location'])
data['Category'] = LabelEncoder().fit_transform(data['Category'])
data['budget_group'] =
LabelEncoder().fit_transform(data['budget_group'])

data = pd.get_dummies(data, columns=['Experience Level', 'Job Type'],
prefix=['Experience', 'JobType'])

```

Standardizing numeric features

```

from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

scaler = StandardScaler()
numeric_features = ['cost_per_skill', 'proposal_to_budget_ratio',
'client_spending_frequency', 'Client Spent', 'Proposals']
data[numeric_features] = scaler.fit_transform(data[numeric_features])

data.head()

{"summary": "{\n  \"name\": \"data\",\n  \"rows\": 1237,\n  \"fields\": [\n    {\n      \"column\": \"Job Title\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 1153,\n        \"samples\": [\n          \"high quality product photography vitamin bottle spoon that can be used in digital advertising\",\n          \"google analytics reporting specialist needed\",\n          \"prospect generation for niche software\",\n          \"semantic_type\": \"\", \n          \"description\": \"\" \n        ],\n        \"column\": \"Category\",\n        \"properties\": {\n          \"dtype\": \"number\",\n          \"std\": 2,\n          \"min\": 0,\n          \"max\": 9,\n          \"num_unique_values\": 10,\n          \"samples\": [\n            6,\n            3,\n            0\n          ],\n        }\n      }\n    }\n  ]\n}"

```

```

{"semantic_type": "",\n      "description": ""\n    },\n    {\n      "column": "Description",\n      "properties": {\n        "dtype": "string",\n        "num_unique_values": 1145,\n        "samples": [\n          "we are seeking a skilled cold calling specialist to help us introduce our services and set up meetings with potential clients the ideal candidate will have experience in making outbound calls engaging prospects and securing appointments you will be responsible for creating a positive first impression and ensuring our message resonates with the target audience if you have excellent communication skills and a track record in successful cold calling we want to hear from you relevant skills cold calling appointment setting lead generation excellent communication skills persuasion and negotiation skills will pay per meeting set up 10 usd only apply if ok",\n          "we are seeking an experienced react developer to build a dynamic single page application spa that provides a seamless user experience the ideal candidate should be comfortable with componentbased architecture and have a strong understanding of state management and api integration your role will involve collaborating with our design and backend teams to transform wireframes into highquality code if you have a passion for developing engaging web applications and a keen eye for detail we want to hear from you",\n          "we are seeking a talented graphic designer and writer to create a visually appealing tweet graphic for our recently accepted academic paper at a prestigious conference the ideal candidate will transform complex research findings into an engaging and informative design that captures attention on social media a quick turnaround and creativity are essential as well as understanding academic themes if you have a passion for science communication and experience in creating impactful visuals we would love to hear from you"\n        ],\n      },\n    },\n    {\n      "column": "Skills",\n      "properties": {\n        "dtype": "string",\n        "num_unique_values": 1137,\n        "samples": [\n          "PyTorch,Cloud Computing",\n          "Social Media Marketing,Communications,Email Communication,Instagram,Facebook,Administrative Support",\n          "Data Entry,Amazon PPC,SEO Backlinking,Amazon FBA,PPC Campaign Setup & Management,Dropshipping"\n        ],\n      },\n    },\n    {\n      "semantic_type": "",\n      "description": ""\n    },\n    {\n      "column": "Budget",\n      "properties": {\n        "dtype": "number",\n        "std": 538.4613409087323,\n        "min": 1.0,\n        "max": 3252.5064000000007,\n        "num_unique_values": 274,\n        "samples": [\n          419.04,\n          580.04285625,\n          1807.2126\n        ],\n      },\n    },\n    {\n      "semantic_type": "",\n      "description": ""\n    },\n    {\n      "column": "Client Location",\n      "properties": {\n        "dtype": "number",\n        "std": 2,\n        "min": 0,\n        "max": 9,\n        "num_unique_values": 10,\n        "samples": [\n          9,\n

```



```

3,\n          0\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n          },\n          {\n          \"column\":\n          \"Client Spent\",\n          \"properties\": {\n          \"dtype\":\n          \"number\",\n          \"std\": 1.000404448954858,\n          \"min\": -\n          1.0462814966831464,\n          \"max\": 1.5495404752625996,\n          \"num_unique_values\": 5,\n          \"samples\": [\n          -\n          0.31412658151896167,\n          0.8839450978406135,\n          -\n          0.9131624211987492\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n          },\n          {\n          \"column\":\n          \"Proposals\",\n          \"properties\": {\n          \"dtype\":\n          \"number\",\n          \"std\": 1.000404448954858,\n          \"min\": -\n          0.9222155728610802,\n          \"max\": 1.468486501259247,\n          \"num_unique_values\": 3,\n          \"samples\": [\n          -\n          1.468486501259247,\n          -0.6103848675410375,\n          -\n          0.9222155728610802\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n          },\n          {\n          \"column\":\n          \"cost_per_skill\",\n          \"properties\": {\n          \"dtype\":\n          \"number\",\n          \"std\": 1.0004044489548578,\n          \"min\": -\n          1.0460105670496622,\n          \"max\": 4.994945804814626,\n          \"num_unique_values\": 274,\n          \"samples\": [\n          -\n          0.26933628410986954,\n          0.029790062481909362,\n          2.3097421741756237\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n          },\n          {\n          \"column\":\n          \"proposal_to_budget_ratio\",\n          \"properties\": {\n          \"dtype\": \"number\",\n          \"std\": 1.000404448954858,\n          \"min\": -2.361938385436965,\n          \"max\": 5.5008620488304025,\n          \"num_unique_values\": 274,\n          \"samples\": [\n          -\n          2.358351671561626,\n          0.23772463709810174,\n          3.7365960507253244\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n          },\n          {\n          \"column\":\n          \"client_spending_frequency\",\n          \"properties\": {\n          \"dtype\": \"number\",\n          \"std\": 1.000404448954858,\n          \"min\": -1.3307224103062174,\n          \"max\": 2.0541382212657497,\n          \"num_unique_values\": 10,\n          \"samples\": [\n          -\n          1.3307224103062174,\n          -0.6996466993351728,\n          2.0541382212657497\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n          },\n          {\n          \"column\":\n          \"budget_group\",\n          \"properties\": {\n          \"dtype\":\n          \"number\",\n          \"std\": 0,\n          \"min\": 0,\n          \"max\": 2,\n          \"num_unique_values\": 3,\n          \"samples\":\n          [\n          1,\n          2,\n          0\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n          },\n          {\n          \"column\": \"Experience_entry\",\n          \"properties\": {\n          \"dtype\": \"boolean\",\n          \"num_unique_values\": 2,\n          \"samples\": [\n          true,\n          false\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n          },\n          {\n          \"column\":\n          \"Experience_expert\",\n          \"properties\": {\n          \"dtype\":\n          \"boolean\",\n          \"num_unique_values\": 2,\n          \"samples\":

```



```
[\\n      true,\\n      false\\n    ],\\n
\\\"semantic_type\\\": \\\"\\\",\\n    \\\"description\\\": \\\"\\\"\\n  }\\n
n  },\\n  {\\n    \\\"column\\\": \\\"Experience_intermediate\\\",\\n
\\\"properties\\\": {\\n      \\\"dtype\\\": \\\"boolean\\\",\\n
\\\"num_unique_values\\\": 2,\\n      \\\"samples\\\": [\\n        false,\\n
true\\n      ],\\n      \\\"semantic_type\\\": \\\"\\\",\\n
\\\"description\\\": \\\"\\\"\\n    }\\n  },\\n  {\\n    \\\"column\\\":
\\\"JobType_fixed price\\\",\\n    \\\"properties\\\": {\\n      \\\"dtype\\\":
\\\"boolean\\\",\\n      \\\"num_unique_values\\\": 2,\\n      \\\"samples\\\":
[\\n        true,\\n        false\\n      ],\\n
\\\"semantic_type\\\": \\\"\\\",\\n      \\\"description\\\": \\\"\\\"\\n    }\\n
n  },\\n  {\\n    \\\"column\\\": \\\"JobType_hourly\\\",\\n
\\\"properties\\\": {\\n      \\\"dtype\\\": \\\"boolean\\\",\\n
\\\"num_unique_values\\\": 2,\\n      \\\"samples\\\": [\\n        false,\\n
true\\n      ],\\n      \\\"semantic_type\\\": \\\"\\\",\\n
\\\"description\\\": \\\"\\\"\\n    }\\n  }\\n ]\\n
n}\\\", \"type\": \"dataframe\", \"variable_name\": \"data\"}
```

TF-IDF Values

```
print(skills_tfidf)

(0, 1)    0.2635513109509214
(0, 30)   0.769857648548674
(0, 3)    0.23853736317776414
(0, 471)  0.19096941631840683
(0, 246)  0.34963589337077766
(0, 318)  0.34963589337077766
(1, 1)    0.49569912217031564
(1, 30)   0.4826609275245913
(1, 15)   0.3056976941329782
(1, 19)   0.46418307016133253
(1, 163)  0.4608728745522556
(2, 1)    0.3111031700593772
(2, 30)   0.6058406718949467
(2, 471)  0.45085103607857896
(2, 298)  0.3317222652505331
(2, 216)  0.32405223938180683
(2, 161)  0.21189558913137396
(2, 358)  0.27010365661929964
(3, 1)    0.5835052803617784
(3, 30)   0.5681575521090927
(3, 471)  0.4228082281485899
(3, 161)  0.3974314858955895
(4, 1)    0.2689165651206993
(4, 141)  0.660443604899216
(4, 157)  0.3271105289842065
:
:
(1232, 292)  0.4695188517480098
```

```
(1232, 248) 0.4846318034346623
(1232, 27) 0.5075354322940809
(1233, 141) 0.2876291507534303
(1233, 33) 0.28047157245933635
(1233, 292) 0.2761975619517432
(1233, 248) 0.2850878554814015
(1233, 481) 0.17825577198031622
(1233, 488) 0.3755542482418857
(1233, 192) 0.32044727611071444
(1233, 416) 0.38842355036015563
(1233, 367) 0.34844784670877754
(1233, 293) 0.36503926278415766
(1234, 143) 0.23501052149012547
(1234, 248) 0.7564661264819921
(1234, 436) 0.4379275613881779
(1234, 418) 0.4251452754657795
(1235, 143) 0.23501052149012547
(1235, 248) 0.7564661264819921
(1235, 436) 0.4379275613881779
(1235, 418) 0.4251452754657795
(1236, 143) 0.23501052149012547
(1236, 248) 0.7564661264819921
(1236, 436) 0.4379275613881779
(1236, 418) 0.4251452754657795
```

```
print(title_tfidf)
```

```
(0, 113) 0.4423029963142705
(0, 27) 0.5359888992106934
(0, 304) 0.4001265979478083
(0, 276) 0.5974802632631815
(1, 464) 0.4937739125369053
(1, 263) 0.4083358532051702
(1, 28) 0.5487862843746614
(1, 371) 0.5369197038361098
(2, 27) 0.5081078660474161
(2, 29) 0.5664005766148651
(2, 186) 0.1554457955773931
(2, 472) 0.31136766932218196
(2, 388) 0.5476345153560659
(3, 29) 0.58274631616161
(3, 186) 0.3198636035186519
(3, 266) 0.4001588446440787
(3, 497) 0.4001588446440787
(3, 474) 0.4876882244327495
(4, 186) 0.3262175073315062
(4, 297) 0.457427715415322
(4, 349) 0.827249673968106
(5, 219) 1.0
(6, 27) 0.6830955356177616
```

```
(6, 2) 0.5739821487132658
(6, 487) 0.45158053786405106
:
(1233, 446) 0.22795149783320015
(1233, 346) 0.33370115641241904
(1233, 131) 0.25246429145259414
(1233, 130) 0.25851191655364736
(1233, 301) 0.4341402901582057
(1233, 13) 0.3750755240878309
(1233, 244) 0.4341402901582057
(1233, 469) 0.4341402901582057
(1234, 186) 0.20631475672006003
(1234, 25) 0.31678160040598924
(1234, 239) 0.6612370729139672
(1234, 31) 0.4841008151494541
(1234, 133) 0.43069196995017855
(1235, 186) 0.20631475672006003
(1235, 25) 0.31678160040598924
(1235, 239) 0.6612370729139672
(1235, 31) 0.4841008151494541
(1235, 133) 0.43069196995017855
(1236, 186) 0.16363268720577165
(1236, 297) 0.22944852619372913
(1236, 25) 0.25124632554573334
(1236, 162) 0.41214282782265416
(1236, 239) 0.524441396442637
(1236, 132) 0.31396765891649336
(1236, 222) 0.5603363469660436
```

```
print(description_tfidf)
```

```
(0, 435) 0.08267257165827567
(0, 20) 0.07022177579996874
(0, 172) 0.07879872959297264
(0, 430) 0.08349244180523901
(0, 68) 0.09301624107008248
(0, 22) 0.04058735006808711
(0, 485) 0.06074419748234882
(0, 303) 0.15071909258953853
(0, 286) 0.09383682551207949
(0, 276) 0.1942788153355925
(0, 153) 0.07944675049460093
(0, 250) 0.2670659528710847
(0, 469) 0.06316707709079676
(0, 443) 0.04663621982024985
(0, 455) 0.07071763934829492
(0, 431) 0.2764158979636849
(0, 39) 0.06760742701817177
(0, 17) 0.06190509466589381
(0, 463) 0.08126986501376407
```

```

(0, 282) 0.06989737560666467
(0, 194) 0.06708938806193131
(0, 231) 0.11701401030832272
(0, 400) 0.2292599151017186
(0, 26) 0.14335269211281682
(0, 462) 0.2733770288948277
:      :
(1236, 495) 0.1682731851918314
(1236, 470) 0.16133884235382115
(1236, 377) 0.10994693859309039
(1236, 176) 0.10499080869041376
(1236, 302) 0.10143877527495022
(1236, 154) 0.1007344449006893
(1236, 408) 0.12025269855488868
(1236, 109) 0.1475274923308567
(1236, 255) 0.14333172590202023
(1236, 195) 0.1363356519556844
(1236, 341) 0.14729870497358255
(1236, 155) 0.1533167766420163
(1236, 37) 0.181787600053512
(1236, 92) 0.17467940677358204
(1236, 144) 0.155386587155295
(1236, 204) 0.11486473712751674
(1236, 69) 0.11789964042447983
(1236, 236) 0.22710152248038445
(1236, 158) 0.21527412211008945
(1236, 223) 0.22710152248038445
(1236, 46) 0.20238569466845732
(1236, 28) 0.23968592758885834
(1236, 457) 0.1914004110481158
(1236, 117) 0.17656573667139186
(1236, 373) 0.2362669410396492

```

Machine Learning Model

Machine learning model to predict budget based on various factors

```

from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score
from sklearn.model_selection import train_test_split, cross_val_score,
KFold
import numpy as np
import pandas as pd
from scipy.sparse import hstack, csr_matrix

data_numeric = data.drop(columns=['Budget', 'Job Title',

```

```

'Description',
'Skills'])).select_dtypes(exclude=['object']).astype('float64')
data_numeric_sparse = csr_matrix(data_numeric.values) # Convert to
sparse format

X_tfidf = hstack([skills_tfidf, title_tfidf, description_tfidf,
data_numeric_sparse])
y = data['Budget']

X_train, X_test, y_train, y_test = train_test_split(X_tfidf, y,
test_size=0.2, random_state=42)

rf_model = RandomForestRegressor(
    n_estimators=50,
    max_depth=5,
    max_features='sqrt',
    min_samples_split=15,
    min_samples_leaf=10,
    random_state=42
)

# Cross-validation for Random Forest
kf = KFold(n_splits=5, shuffle=True, random_state=42)
rf_cv_scores = cross_val_score(rf_model, X_train, y_train, cv=kf,
scoring='r2')

print("\n### Random Forest Cross-Validation Results ###")
print(f"Mean R²: {rf_cv_scores.mean():.2f}")
print(f"Std Dev: {rf_cv_scores.std():.2f}")

rf_model.fit(X_train, y_train)
y_pred_rf_log = rf_model.predict(X_test)

mse_rf_log = mean_squared_error(np.expm1(y_test),
np.expm1(y_pred_rf_log))
rmse_rf_log = np.sqrt(mse_rf_log)
r2_rf_log = r2_score(np.expm1(y_test), np.expm1(y_pred_rf_log))

print("\n### Random Forest Results ###")
print(f"MSE: {mse_rf_log:.2f}")
print(f"RMSE: {rmse_rf_log:.2f}")
print(f"R² Score: {r2_rf_log:.2f}")

### Random Forest Cross-Validation Results ###
Mean R²: 0.95
Std Dev: 0.01

### Random Forest Results ###
MSE: 1468481.98

```

RMSE: 1211.81
R² Score: 0.90

Machine learning model to predict skill based on description and title given by user

```
def simplify_skills(skill):  
    if "WordPress" in skill:  
        return "WordPress"  
    elif "Graphic Design" in skill or "Adobe" in skill:  
        return "Graphic Design"  
    elif "Data Entry" in skill or "Administrative Support" in skill:  
        return "Administrative Support"  
    elif "Video Editing" in skill or "Motion Graphics" in skill:  
        return "Video Editing"  
    elif "Web Development" in skill or "Web Design" in skill:  
        return "Web Development"  
    elif "3D" in skill:  
        return "3D Design & Modeling"  
    elif "SEO" in skill or "Search Engine Optimization" in skill:  
        return "SEO & Digital Marketing"  
    elif "Accounting" in skill or "Bookkeeping" in skill:  
        return "Accounting & Finance"  
    elif "Animation" in skill:  
        return "Animation"  
    elif "AI" in skill or "Machine Learning" in skill:  
        return "AI & Machine Learning"  
    elif "API" in skill:  
        return "API Development & Integration"  
    elif "Content Writing" in skill or "Writing" in skill:  
        return "Writing & Content Creation"  
    elif "Presentation Design" in skill or "Microsoft PowerPoint" in  
skill:  
        return "Presentation Design"  
    elif "Lead Generation" in skill or "Sales" in skill:  
        return "Sales & Lead Generation"  
    elif "Architecture" in skill or "Rendering" in skill:  
        return "Architecture & Rendering"  
    else:  
        return "Other"  
  
data['Simplified Skills'] = data['Skills'].apply(simplify_skills)  
simplified_skill_counts = data['Simplified Skills'].value_counts()  
print(simplified_skill_counts)
```

Simplified Skills	
Other	824
Graphic Design	119
Administrative Support	100

```
WordPress      86
Video Editing   63
Web Development 58
Name: count, dtype: int64
```

```
from sklearn.metrics import classification_report, accuracy_score

label_encoder = LabelEncoder()
data['Simplified Skills Encoded'] =
label_encoder.fit_transform(data['Simplified Skills'])

# Vectorize the Description column using TF-IDF
tfidf = TfidfVectorizer(max_features=500, stop_words='english')
X = tfidf.fit_transform(data['Description'])
y = data['Simplified Skills Encoded']

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

logistic_model = LogisticRegression(max_iter=1000, random_state=42)
logistic_model.fit(X_train, y_train)

y_pred = logistic_model.predict(X_test)

# Get the unique classes from the true and predicted values
unique_classes = sorted(set(y_test).union(set(y_pred)))

classification_rep = classification_report(
    y_test,
    y_pred,
    labels=unique_classes, # Specify the correct set of labels
    target_names=label_encoder.classes_[unique_classes], # Ensure
target_names matches the labels
    zero_division=0
)

accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")
print("Classification Report:")
print(classification_rep)
```

Accuracy: 0.70

Classification Report:

	precision	recall	f1-score	support
Administrative Support	0.58	0.35	0.44	20
Graphic Design	0.67	0.33	0.44	24
Other	0.71	0.93	0.81	162
Video Editing	0.50	0.17	0.25	12

Web Development	0.00	0.00	0.00	16
WordPress	0.88	0.44	0.58	16
accuracy			0.70	250
macro avg	0.56	0.37	0.42	250
weighted avg	0.65	0.70	0.65	250

Predict Skills

```
def predict_skill_interactive(model, vectorizer, label_encoder):
    job_description = input("Enter the job description: ")

    vectorized_desc = vectorizer.transform([job_description])
    predicted_label = model.predict(vectorized_desc)[0]
    predicted_skill =
label_encoder.inverse_transform([predicted_label])[0]

    print(f"Predicted Skill: {predicted_skill}")

predict_skill_interactive(logistic_model, tfidf, label_encoder)

Enter the job description: Wordpress developer required
Predicted Skill: Wordpress
```

Deep Learning Model

Why Using ANN?

ANN is the best choice because our problem involves tabular data (numerical and categorical) with Word2Vec embeddings for text features, which ANNs excel at handling. RNN and CNN are more specialized models that excel in tasks involving sequential or spatial data. Since our dataset is not sequential (like time series or ordered text), and does not involve images or grid-like data, RNNs and CNNs would not be as effective as ANN for this particular problem.

Why we used relu in hidden layers ?

We used ReLU because it's fast and efficient, helping the model learn quicker without getting stuck like other activation functions (like sigmoid or tanh). ReLU avoids the vanishing gradient problem, allowing the model to train better in deep networks. It also creates sparsity by turning negative values to zero, which can help the model generalize well to new data.

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from sklearn.model_selection import train_test_split
```



```

target = ['Budget']
X = data.drop(columns=['Budget', 'Job Title', 'Skills', 'Description'])
y = data[target]

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

model = Sequential()

model.add(Dense(512, input_dim=X_train.shape[1], activation='relu'))

# 3 Hidden layers are used which is typical required for small data to
avoid underfit or overfit
model.add(Dense(256, activation='relu'))
model.add(Dropout(0.3))
model.add(Dense(128, activation='relu'))
model.add(Dropout(0.3))
model.add(Dense(64, activation='relu'))

# Output layer
model.add(Dense(1, activation='linear'))

model.compile(optimizer='adam', loss='mse', metrics=['mae'])
model.summary()
ann_model = model.fit(
    X_train, y_train,
    validation_data=(X_test, y_test),
    epochs=50,
    batch_size=32,
    verbose=1
)

# ANN Metrics
y_pred_ann_log = ann_model.predict(X_test).flatten()
mse_ann_log = mean_squared_error(np.expm1(y_test),
np.expm1(y_pred_ann_log))
rmse_ann_log = np.sqrt(mse_ann_log)
r2_ann_log = r2_score(np.expm1(y_test), np.expm1(y_pred_ann_log))

print("\n### ANN Results ###")
print(f"MSE: {mse_ann_log:.2f}")
print(f"RMSE: {rmse_ann_log:.2f}")
print(f"R2 Score: {r2_ann_log:.2f}")

Epoch 1/50
100/100 [=====] - 1s 4ms/step - loss: 36.2317
- mae: 5.6419 - val_loss: 11.6554 - val_mae: 2.8092
Epoch 2/50
100/100 [=====] - 0s 2ms/step - loss: 9.1080
- mae: 2.3358 - val_loss: 6.0908 - val_mae: 1.7589

```

```
Epoch 3/50
100/100 [=====] - 0s 2ms/step - loss: 6.1206
- mae: 1.8356 - val_loss: 4.2837 - val_mae: 1.4581
Epoch 4/50
100/100 [=====] - 0s 2ms/step - loss: 5.3715
- mae: 1.6652 - val_loss: 2.8108 - val_mae: 1.2172
Epoch 5/50
100/100 [=====] - 0s 2ms/step - loss: 4.0630
- mae: 1.4959 - val_loss: 2.3891 - val_mae: 1.0675
Epoch 6/50
100/100 [=====] - 0s 2ms/step - loss: 4.0652
- mae: 1.4699 - val_loss: 1.5785 - val_mae: 0.8910
Epoch 7/50
100/100 [=====] - 0s 2ms/step - loss: 3.5528
- mae: 1.3863 - val_loss: 1.1892 - val_mae: 0.7469
Epoch 8/50
100/100 [=====] - 0s 2ms/step - loss: 3.4284
- mae: 1.3352 - val_loss: 1.1958 - val_mae: 0.7329
Epoch 9/50
100/100 [=====] - 0s 2ms/step - loss: 3.0499
- mae: 1.2669 - val_loss: 1.0361 - val_mae: 0.6761
Epoch 10/50
100/100 [=====] - 0s 2ms/step - loss: 2.7010
- mae: 1.2040 - val_loss: 0.9013 - val_mae: 0.6017
Epoch 11/50
100/100 [=====] - 0s 2ms/step - loss: 2.5355
- mae: 1.1606 - val_loss: 0.9517 - val_mae: 0.6301
Epoch 12/50
100/100 [=====] - 0s 2ms/step - loss: 2.7883
- mae: 1.2192 - val_loss: 0.7957 - val_mae: 0.5517
Epoch 13/50
100/100 [=====] - 0s 2ms/step - loss: 2.5860
- mae: 1.1621 - val_loss: 0.8269 - val_mae: 0.5468
Epoch 14/50
100/100 [=====] - 0s 2ms/step - loss: 2.5561
- mae: 1.1486 - val_loss: 0.7377 - val_mae: 0.5298
Epoch 15/50
100/100 [=====] - 0s 2ms/step - loss: 2.6166
- mae: 1.2017 - val_loss: 0.6493 - val_mae: 0.4804
Epoch 16/50
100/100 [=====] - 0s 2ms/step - loss: 2.5330
- mae: 1.1475 - val_loss: 0.7741 - val_mae: 0.5865
Epoch 17/50
100/100 [=====] - 0s 2ms/step - loss: 2.2972
- mae: 1.1088 - val_loss: 0.6293 - val_mae: 0.4646
Epoch 18/50
100/100 [=====] - 0s 2ms/step - loss: 2.3393
- mae: 1.1388 - val_loss: 0.6478 - val_mae: 0.4946
Epoch 19/50
```

```
100/100 [=====] - 0s 2ms/step - loss: 2.2857
- mae: 1.1064 - val_loss: 0.7511 - val_mae: 0.5909
Epoch 20/50
100/100 [=====] - 0s 2ms/step - loss: 2.6743
- mae: 1.1952 - val_loss: 0.5341 - val_mae: 0.4190
Epoch 21/50
100/100 [=====] - 0s 2ms/step - loss: 2.2542
- mae: 1.1119 - val_loss: 0.5940 - val_mae: 0.4790
Epoch 22/50
100/100 [=====] - 0s 2ms/step - loss: 2.1285
- mae: 1.0724 - val_loss: 0.6649 - val_mae: 0.5484
Epoch 23/50
100/100 [=====] - 0s 2ms/step - loss: 2.1127
- mae: 1.0644 - val_loss: 0.6246 - val_mae: 0.5196
Epoch 24/50
100/100 [=====] - 0s 2ms/step - loss: 2.2114
- mae: 1.0904 - val_loss: 0.5995 - val_mae: 0.4963
Epoch 25/50
100/100 [=====] - 0s 2ms/step - loss: 2.0753
- mae: 1.0430 - val_loss: 0.5328 - val_mae: 0.4722
Epoch 26/50
100/100 [=====] - 0s 2ms/step - loss: 2.0835
- mae: 1.0721 - val_loss: 0.5898 - val_mae: 0.5269
Epoch 27/50
100/100 [=====] - 0s 2ms/step - loss: 2.0207
- mae: 1.0617 - val_loss: 0.6523 - val_mae: 0.5781
Epoch 28/50
100/100 [=====] - 0s 2ms/step - loss: 1.9335
- mae: 1.0346 - val_loss: 0.5222 - val_mae: 0.4545
Epoch 29/50
100/100 [=====] - 0s 2ms/step - loss: 2.0920
- mae: 1.0741 - val_loss: 0.5086 - val_mae: 0.4643
Epoch 30/50
100/100 [=====] - 0s 2ms/step - loss: 1.8731
- mae: 1.0066 - val_loss: 0.5302 - val_mae: 0.4921
Epoch 31/50
100/100 [=====] - 0s 2ms/step - loss: 1.8069
- mae: 0.9876 - val_loss: 0.4674 - val_mae: 0.4359
Epoch 32/50
100/100 [=====] - 0s 2ms/step - loss: 2.1102
- mae: 1.0677 - val_loss: 0.5472 - val_mae: 0.5049
Epoch 33/50
100/100 [=====] - 0s 2ms/step - loss: 1.9379
- mae: 1.0345 - val_loss: 0.4062 - val_mae: 0.3823
Epoch 34/50
100/100 [=====] - 0s 2ms/step - loss: 1.7770
- mae: 0.9877 - val_loss: 0.4707 - val_mae: 0.4561
Epoch 35/50
100/100 [=====] - 0s 2ms/step - loss: 1.7568
```

```
- mae: 0.9725 - val_loss: 0.4138 - val_mae: 0.3950
Epoch 36/50
100/100 [=====] - 0s 2ms/step - loss: 1.7949
- mae: 0.9944 - val_loss: 0.4451 - val_mae: 0.4448
Epoch 37/50
100/100 [=====] - 0s 2ms/step - loss: 1.7805
- mae: 0.9857 - val_loss: 0.3899 - val_mae: 0.3855
Epoch 38/50
100/100 [=====] - 0s 2ms/step - loss: 1.7421
- mae: 0.9853 - val_loss: 0.5731 - val_mae: 0.5405
Epoch 39/50
100/100 [=====] - 0s 2ms/step - loss: 1.7717
- mae: 0.9952 - val_loss: 0.3342 - val_mae: 0.3295
Epoch 40/50
100/100 [=====] - 0s 2ms/step - loss: 1.6676
- mae: 0.9657 - val_loss: 0.3743 - val_mae: 0.3918
Epoch 41/50
100/100 [=====] - 0s 2ms/step - loss: 1.6402
- mae: 0.9354 - val_loss: 0.5000 - val_mae: 0.5099
Epoch 42/50
100/100 [=====] - 0s 2ms/step - loss: 1.5892
- mae: 0.9315 - val_loss: 0.4963 - val_mae: 0.5049
Epoch 43/50
100/100 [=====] - 0s 2ms/step - loss: 1.5767
- mae: 0.9413 - val_loss: 0.3729 - val_mae: 0.4034
Epoch 44/50
100/100 [=====] - 0s 2ms/step - loss: 1.6161
- mae: 0.9409 - val_loss: 0.3169 - val_mae: 0.3370
Epoch 45/50
100/100 [=====] - 0s 2ms/step - loss: 1.4932
- mae: 0.9107 - val_loss: 0.3549 - val_mae: 0.3609
Epoch 46/50
100/100 [=====] - 0s 2ms/step - loss: 1.3817
- mae: 0.8574 - val_loss: 0.3368 - val_mae: 0.3696
Epoch 47/50
100/100 [=====] - 0s 2ms/step - loss: 1.6092
- mae: 0.9452 - val_loss: 0.3778 - val_mae: 0.4266
Epoch 48/50
100/100 [=====] - 0s 2ms/step - loss: 1.4094
- mae: 0.8806 - val_loss: 0.2848 - val_mae: 0.3093
Epoch 49/50
100/100 [=====] - 0s 2ms/step - loss: 1.5798
- mae: 0.9454 - val_loss: 0.2947 - val_mae: 0.3325
Epoch 50/50
100/100 [=====] - 0s 2ms/step - loss: 1.5288
- mae: 0.9287 - val_loss: 0.3274 - val_mae: 0.3841
8/8 [=====] - 0s 1ms/step
```

ANN Results

MSE: 3261902.41

RMSE: 1806.07
R² Score: 0.78

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import classification_report, accuracy_score
from imblearn.over_sampling import SMOTE
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout
from tensorflow.keras.preprocessing.sequence import pad_sequences
from sklearn.feature_extraction.text import TfidfVectorizer
import tensorflow as tf

def merge_classes(skill):
    if skill in ["Administrative Support", "WordPress"]:
        return "Support"
    elif skill in ["Graphic Design", "Video Editing"]:
        return "Design"
    elif skill in ["Web Development", "Programming"]:
        return "Development"
    else:
        return "Other"

data['Merged Skills'] = data['Simplified Skills'].apply(merge_classes)
merged_skill_counts = data['Merged Skills'].value_counts()
print("Class distribution after merging:")
print(merged_skill_counts)

label_encoder = LabelEncoder()
y = label_encoder.fit_transform(data['Merged Skills'])

tfidf = TfidfVectorizer(max_features=500, stop_words='english')
X = tfidf.fit_transform(data['Description'])

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42, stratify=y)

# Apply SMOTE for balancing
smote = SMOTE(random_state=42)
X_train_balanced, y_train_balanced = smote.fit_resample(X_train,
y_train)

# Convert sparse matrices to dense for LSTM input
X_train_balanced = X_train_balanced.toarray()
X_test = X_test.toarray()

max_length = max(len(seq) for seq in X_train_balanced)
```

```

X_train_balanced = pad_sequences(X_train_balanced, maxlen=max_length,
padding='post')
X_test = pad_sequences(X_test, maxlen=max_length, padding='post')
vocab_size = max(X_train_balanced.max(), X_test.max()) + 1
print(f"Adjusted Vocabulary Size: {vocab_size}")
y_train_balanced_onehot = np.eye(len(label_encoder.classes_))
[y_train_balanced]
y_test_onehot = np.eye(len(label_encoder.classes_))[y_test]

embedding_dim = 100
model = Sequential([
    Embedding(input_dim=vocab_size, output_dim=embedding_dim),
    LSTM(128, return_sequences=True),
    Dropout(0.3),
    LSTM(64),
    Dropout(0.3),
    Dense(64, activation='relu'),
    Dense(len(label_encoder.classes_), activation='softmax')
])

model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])

history = model.fit(
    X_train_balanced,
    y_train_balanced_onehot,
    validation_split=0.2,
    epochs=10,
    batch_size=32,
    verbose=1
)

y_pred_prob = model.predict(X_test)
y_pred = np.argmax(y_pred_prob, axis=1)

accuracy = accuracy_score(y_test, y_pred)
classification_rep = classification_report(y_test, y_pred,
target_names=label_encoder.classes_)
print(f"Class distribution after SMOTE:\n{np.bincount(y_train_balanced)}")

print(f"Accuracy: {accuracy:.2f}")
print("Classification Report:")
print(classification_rep)

def predict_skill(description):
    description_sequence = tfidf.transform([description]).toarray()
    description_sequence = pad_sequences(description_sequence,

```

```

maxlen=max_length, padding='post')
    prediction = model.predict(description_sequence)
    predicted_label = np.argmax(prediction, axis=1)
    return label_encoder.inverse_transform(predicted_label)[0]

user_input = input("Enter job description: ")
predicted_skill = predict_skill(user_input)
print(f"Predicted Skill: {predicted_skill}")

Class distribution after merging:
Merged Skills
Other          824
Support        186
Design         182
Development     58
Name: count, dtype: int64
Adjusted Vocabulary Size: 2
Epoch 1/10
66/66 [=====] - 59s 844ms/step - loss: 1.3001
- accuracy: 0.3112 - val_loss: 2.5726 - val_accuracy: 0.0000e+00
Epoch 2/10
66/66 [=====] - 54s 815ms/step - loss: 1.2878
- accuracy: 0.3036 - val_loss: 2.3170 - val_accuracy: 0.0000e+00
Epoch 3/10
66/66 [=====] - 54s 818ms/step - loss: 1.2826
- accuracy: 0.2927 - val_loss: 2.5297 - val_accuracy: 0.0000e+00
Epoch 4/10
66/66 [=====] - 54s 813ms/step - loss: 1.2850
- accuracy: 0.3083 - val_loss: 2.5817 - val_accuracy: 0.0000e+00
Epoch 5/10
66/66 [=====] - 54s 813ms/step - loss: 1.2820
- accuracy: 0.3074 - val_loss: 2.7406 - val_accuracy: 0.0341
Epoch 6/10
66/66 [=====] - 54s 812ms/step - loss: 1.2842
- accuracy: 0.3069 - val_loss: 2.6857 - val_accuracy: 0.0000e+00
Epoch 7/10
66/66 [=====] - 54s 818ms/step - loss: 1.2795
- accuracy: 0.2989 - val_loss: 2.4925 - val_accuracy: 0.0000e+00
Epoch 8/10
66/66 [=====] - 54s 814ms/step - loss: 1.2792
- accuracy: 0.2974 - val_loss: 2.2868 - val_accuracy: 0.0341
Epoch 9/10
66/66 [=====] - 54s 821ms/step - loss: 1.2810
- accuracy: 0.3245 - val_loss: 2.6171 - val_accuracy: 0.0341
Epoch 10/10
66/66 [=====] - 53s 804ms/step - loss: 1.2831
- accuracy: 0.2922 - val_loss: 2.5557 - val_accuracy: 0.0000e+00
8/8 [=====] - 3s 219ms/step

```

```

/usr/local/lib/python3.10/dist-packages/sklearn/metrics/_classification.py:1531: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is",
len(result))
/usr/local/lib/python3.10/dist-packages/sklearn/metrics/_classification.py:1531: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is",
len(result))
/usr/local/lib/python3.10/dist-packages/sklearn/metrics/_classification.py:1531: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is",
len(result))

```

Class distribution after SMOTE:

[659 659 659 659]

Accuracy: 0.14

Classification Report:

	precision	recall	f1-score	support
Design	0.14	1.00	0.25	36
Development	0.00	0.00	0.00	12
Other	0.00	0.00	0.00	165
Support	0.00	0.00	0.00	37
accuracy			0.14	250
macro avg	0.04	0.25	0.06	250
weighted avg	0.02	0.14	0.04	250

Enter job description: video editing

1/1 [=====] - 0s 85ms/step

Predicted Skill: Design

pip install imblearn

Collecting imblearn

Downloading imblearn-0.0-py2.py3-none-any.whl.metadata (355 bytes)

Collecting imbalanced-learn (from imblearn)

Downloading imbalanced_learn-0.12.4-py3-none-any.whl.metadata (8.3 kB)

Requirement already satisfied: numpy>=1.17.3 in

/usr/local/lib/python3.10/dist-packages (from imbalanced-learn->imblearn) (1.26.4)

Requirement already satisfied: scipy>=1.5.0 in

/usr/local/lib/python3.10/dist-packages (from imbalanced-learn-


```

>imblearn) (1.13.1)
Requirement already satisfied: scikit-learn>=1.0.2 in
/usr/local/lib/python3.10/dist-packages (from imbalanced-learn-
>imblearn) (1.5.2)
Requirement already satisfied: joblib>=1.1.1 in
/usr/local/lib/python3.10/dist-packages (from imbalanced-learn-
>imblearn) (1.4.2)
Requirement already satisfied: threadpoolctl>=2.0.0 in
/usr/local/lib/python3.10/dist-packages (from imbalanced-learn-
>imblearn) (3.5.0)
Downloading imblearn-0.0-py2.py3-none-any.whl (1.9 kB)
Downloading imbalanced_learn-0.12.4-py3-none-any.whl (258 kB)
_____ 258.3/258.3 kB 4.6 MB/s eta
0:00:00
balanced-learn, imblearn
Successfully installed imbalanced-learn-0.12.4 imblearn-0.0

```

Machine Vs Deep Learning Model

```

# Compare Results
print("\nComparison of Models")
print(f"Random Forest - MSE: {mse_rf_log:.2f}, RMSE:
{rmse_rf_log:.2f}, R²: {r2_rf_log:.2f}")
print(f"ANN - MSE: {mse_ann_log:.2f}, RMSE: {rmse_ann_log:.2f}, R²:
{r2_ann_log:.2f}")

# Visualize Actual vs Predicted
y_test_original = np.expml(y_test)
y_pred_rf_original = np.expml(y_pred_rf_log)
y_pred_ann_original = np.expml(y_pred_ann_log)

plt.figure(figsize=(12, 6))
plt.scatter(y_test_original, y_pred_rf_original, alpha=0.6,
label='Random Forest Predictions', color='blue')
plt.scatter(y_test_original, y_pred_ann_original, alpha=0.6,
label='ANN Predictions', color='orange')
plt.plot([min(y_test_original), max(y_test_original)],
[min(y_test_original), max(y_test_original)],
color='red', linestyle='--', label='Ideal Predictions')
plt.title("Actual vs Predicted Budget Values")
plt.xlabel("Actual Budget")
plt.ylabel("Predicted Budget")
plt.legend()
plt.grid()
plt.show()

# Visualize ANN Training History
plt.figure(figsize=(10, 5))

```

```

plt.plot(ann_model.history['loss'], label='Training Loss',
color='blue')
plt.plot(ann_model.history['val_loss'], label='Validation Loss',
color='orange')
plt.title("ANN Training History")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.grid()
plt.show()

```

Comparison of Models

Random Forest - MSE: 1468481.98, RMSE: 1211.81, R^2 : 0.90

ANN - MSE: 3261902.41, RMSE: 1806.07, R^2 : 0.78

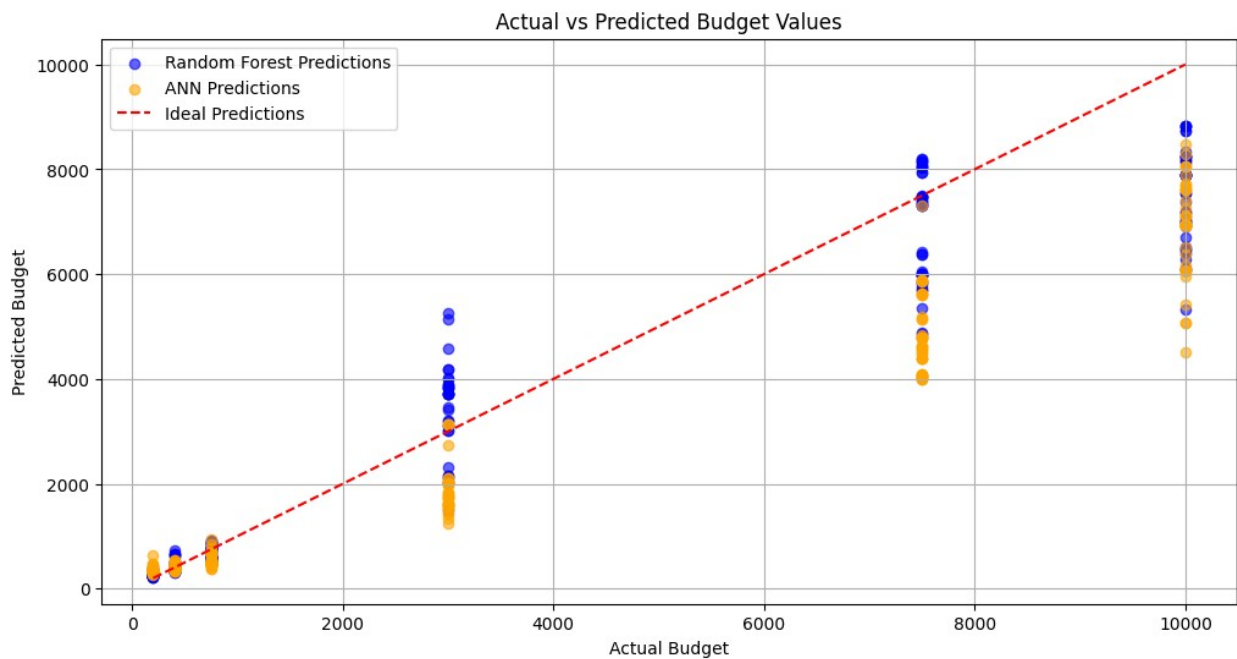




Chart 1 Random Forest

- Random Forest outperforms the ANN model, as evidenced by lower error (MSE, RMSE) and higher R^2 (explaining 90% of the variance).
- Predictions from the Random Forest model are closer to the red dashed line, indicating better accuracy.
- Predictions from the ANN model exhibit more scatter around the red line, reflecting greater error and lower performance.

Chart 2 ANN

- The training loss decreases significantly over the first 5-10 epochs, indicating the model is learning effectively.
- The validation loss follows a similar pattern but stabilizes sooner than the training loss.
- Both curves flatten around 20 epochs, suggesting the model is converging and no longer improving significantly.

Conclusion

The project utilized two models—Random Forest and Artificial Neural Networks (ANN)—to predict job budget values. Based on the results:

Model Performance: The Random Forest model outperformed the ANN model in terms of accuracy, with significantly lower error (MSE: 1,468,481.98 vs. 3,261,902.41) and higher explanatory power (R^2 : 0.90 vs. 0.78).

Predictions from Random Forest aligned closely with the actual budget values, as seen in the chart, while ANN predictions showed more variability.

Training Efficiency: The ANN model demonstrated effective learning with minimal overfitting, as evidenced by the consistent convergence of training and validation loss. However, its predictive accuracy lagged compared to Random Forest.

Model Recommendation: For this project, the Random Forest model is the preferred choice due to its superior predictive performance and ability to handle the dataset's complexity effectively.